

תוכנית בדיקות — בוט האחזקה (takt-bots)

מסמך תכנון לתהליך בדיקה מקצה-לקצה של בוט האחזקה, בשלושה שלבים:
(1) צ'אט-בדיקה באתר → (2) בוט טלגרם → (3) וואטסאפ בפרודקשן.

נכתב לפני מימוש. לאחר אישור — מקודדים שלב-שלב.

0. עקרונות מנחים

- המנוע כבר channel-agnostic.** כל הלוגיקה יושבת ב- `agents/bot_engine/m10010/` מאחורי שתי פונקציות שממופתחות לפי מזהה (`phone`), לא תלויות בערוץ:
- `start_session(phone, name, parsed_data, ..., script_id)` → ההודעה הראשונה
- `process_message(phone, text, msg_type, caption)` → `{text, buttons}` הבא
כל שלושת הערוצים הם מעטפת (**adapter**) דקה סביב אותן פונקציות.
- "פרוירטי" מזויף בבדיקות.** ההסתעפות של הבוט תלויה במקורות חיצוניים (קיום מכשיר, קיום קריאה פתוחה, ניתוב LLM). בבדיקות אנחנו מזייפים אותם דרך ה-`integrations` הפלאגבליים הקיימים — **המנוע רץ ללא שינוי**.
- שכבות:** `api → engine → storage → tools`. קובץ מקור $\geq 500-600$ שורות.
כל דבר חדש נכנס לשכבה הנכונה: API חדש תחת `backend/app/api/`, אחסון תחת `agents/bot_engine/`, `backend, mock + database/maintenance/` תחת `agents/bot_engine/`.
- בלי צימוד ל-urbangroup.** ה-`mock` הוא מימוש עצמאי; אינטגרציית פרוירטי האמיתית נשארת פלאגין נפרד (שלב 3 בלבד).

1. מודל "תנאי הפתיחה" (Scenario)

זה הלב של הבדיקה. סנריו = **מצב התחלתי מלא** שמאפשר להגיע לכל ענף. חמשת התנאים (מהניתוח של המנוע):

#	תנאי	שדה בסכמה	משפיע על
1	תסריט לבדיקה	script_id	נקודת הכניסה
2	זהות הפונה	caller: {phone, name, known, customer_name, customer_number}	לקוח מזוהה מראש?
3	הודעה נכנסת / פורמט	inbound: {text, format: free\ qr\ voice, parsed_data, device_number}	ניתוב מקור + skip_if
4	מכשיר בפריורטי	equipment: {exists, custname, cdes, ...} (מפתח = מספר מכשיר)	check_equipment הצלחה/כישלון
5	קריאה פתוחה קיימת	open_calls: [{DOCNO, CALLSTATUSCODE}] (מפתח = מספר מכשיר)	check_open_service_call
+	ניתוב LLM	+ llm_mode: manual\ live forced_exits: {step_id: exit_index}	צמתי "הוראות לבוט" עם יציאות

סכמת JSON של סנריו

```
{
  "scenario_id": "known-customer-device-exists-no-open-call",
  "name": "לקוח מזוהה, מכשיר קיים, אין קריאה פתוחה",
  "script_id": "maintenance-troubleshoot",
  "caller": {
    "phone": "test-001", "name": "בדיקה", "known": true,
    "customer_name": "חניון רוטשילד", "customer_number": "C1042"
  },
  "inbound": {
    "text": "יש תקלה במחסום", "format": "free",
    "device_number": "", "parsed_data": {}
  },
  "equipment": {
    "12345": { "custname": "C1042", "cdes": "חניון רוטשילד",
    "location": "5 רוטשילד" }
  },
  "open_calls": { "12345": [] },
  "llm_mode": "manual",
  "forced_exits": { "INSTR_1772182740052": 0 },

  "steps": [
    { "send": "12345" },
    { "send": "5 רוטשילד" },
    { "send": "btn_1" }
  ],
  "expect": {
    "final_action": "save_service_call",
    "reach_step": "DONE_1772162664033",
    "contains_text": "פתחנו קריאת שירות"
  }
}
```

- expect + steps משמשים ל-replay/הגרסה (סעיף 2.3). בצ'אט אינטראקטיבי הם ריקים.
- אותה סכמה משמשת גם לצ'אט הידני וגם למבחני ההגרסה — מקור אמת אחד.

2. שלב 1 — צ'אט-בדיקה באתר

API 2.1 — bot-test Backend

קובץ חדש: backend/app/api/bot_test.py (שכבת API בלבד; קורא למנוע ולאחסון).

תיאור	Endpoint
מקבל סנריו; מאפס טשן קודם, טוען את ה-mock מ- equipment / open_calls, מפעיל start_session(script_id, ...) עם ה- caller / inbound, ומחזיר {text, buttons, session}.	POST /api/bot-test/start
ידני מחזיר {needs_route: {step, exits}} → process_message → {tester_id, text} → {text, buttons, session, log}. אם נדרש ניתוב.	POST /api/bot-test/message
{tester_id, step, exit_index} → קובע את היציאה שנבחרה וממשיך.	POST /api/bot-test/route
מוחק את טשן הבדיקה.	POST /api/bot-test/reset

מזהה הבדיקה (tester_id) = "טלפון" מדומה קבוע (למשל test-001). הסשן נשמר ב- troubleshoot_sessions_db הרגיל, כך שהצ'אט משתמש **בדיק** באותו נתיב כמו וואטסאפ. **ה-mock של פריורטי** — קובץ חדש agents/bot_engine/mock_providers.py שחושף בדיוק את הממשק שה-integrations מצפים לו:

```
# equipment reader
fetch_equipment_by_sernum(sernum) -> dict | None      # מ-store["equipment"]
fetch_equipment_by_phone(phone)    -> list           # מ-store["caller"]
# service-call writer
find_open_service_calls(device)    -> list           # מ-store["open_calls"]
create_service_call(data)           -> {"DOCNO": "TEST-<n>"} # לא כותב לפריורטי
append_note_to_service_call(docno, note) -> None
```

ה-store הוא dict במודול, שה- /start מאכלס מהסנריו. מופעל בסביבת dev דרך:

```
EQUIPMENT_READER_ENABLED=true
SERVICE_CALL_WRITER_ENABLED=true
BOT_EQUIPMENT_READER_MODULE=agents.bot_engine.mock_providers
BOT_SERVICE_CALL_WRITER_MODULE=agents.bot_engine.mock_providers
```

מגבלה מודעת: ה-store גלובלי → בודק אחד בכל רגע (מספיק לוקאלית). ריבוי בודקים = שיפור עתידי (מיפתוח לפי tester_id).

2.2 ניתוב LLM — ידני + אמיתי

היום steps._11m_route_exits קורא ישירות ל-OpenAI. נעשה אותו **פלאגבל** (כמו ה-integrations): get_11m_router() שמחזיר את הפותר לפי המצב:

- live — קורא ל-OpenAI האמיתי (התנהגות קיימת). דורש OPENAI_API_KEY.

- **manual** — קורא מ- `forced_exits[step_id]` שבסנריו. אם אין ערך מוגדר → ה-API מחזיר `needs_route` והצ'אט מציג כפתורים ("איזו יציאה?") → הבחירה נשמרת וממשיכים. כך הצ'אט האינטראקטיבי דטרמיניסטי, וה-replay משתמש ב- `forced_exits` מראש.

refactor קטן ומכולל: הוצאת הניתוב מאחורי getter. אין שינוי בהתנהגות הפרודקשן (ברירת מחדל = live).

2.3 Frontend — עמוד "בודק שיחות"

מחליף/מרחיב את `BotDiagnosticsPage` הקיים (שהיום רק **קורא** סשנים ישנים). פריסה בשלושה טורים (RTL):

מפקח ששן	תנאי פתיחה	צ'אט בדיקה
• שלום 🤖	step: STEP_5	... ,תסריט ▼
• 12345 🧑	device: 12345	לקוח מזוהה ☑
• באיזו כתובת 📍	customer: ...	מס' מכשיר
• — לוג	— מכשיר קיים ☑	[כן, מושבתת] [לא]
• קריאה פתוחה ☐	action ✓ ...	...
• פורמט: free ▼	button ✓ ...	
• LLM: [הקלד תשובה...]	[>] ידני ▼	
• [התחל בדיקה ▶]		

- **פאנל תנאי פתיחה** = חמשת התנאים מסעיף 1. "התחל בדיקה" קורא ל- `/start`.
 - **חלון צ'אט** בסגנון וואטסאפ: הודעות בוט/משתמש, כפתורים לחיצים (שולחים `btn.id`), שדה טקסט חופשי. תמיכה ב- `needs_route` (כפתורי יציאה).
 - **מפקח ששן** ח': שדות שנאספו + `session_log` (כבר קיים במנוע) → רואים בזמן אמת לאיזה שלב הגענו ולמה הבוט הסתעף.
- קבצים: `ConditionsPanel.jsx` + `frontend/src/pages/BotTester/BotTesterPage.jsx`
 + `BotTester.css` + `SessionInspector.jsx` + `ChatWindow.jsx` (כל אחד ≥ 500 שורות).
 נוסף פריט ניווט "בודק שיחות" ל- `App.jsx`.

2.4 סנריואים שמורים + Replay (גרסיה)

- **אחסון:** טבלה חדשה `bot_test_scenarios` — `database/maintenance/bot_test_scenarios_db.py`
- מימוש ב- `database/backends/sqlite/`. שומר את סכמת הסנריו המלאה (סעיף 1).
- **API:** `GET/PUT/DELETE /.../{id}`, `GET/POST /api/bot-test/scenarios`
`POST /api/bot-test/scenarios/{id}/run` — מריץ את `steps` דרך המנוע ומחזיר `pass/fail` מול `expect (contains_text / final_action / reach_step)`.
- **"שמור כסנריו"** בצ'אט: לוקח את תנאי הפתיחה + רצף ההודעות שהוקלדו והופך אותם לסנריו.
- **מריץ אצווה:** `POST /api/bot-test/run-all` → מריץ את כל הסנריואים ומחזיר טבלת תוצאות. אפשר להריץ מה-UI (כפתור "הרץ גרסיה") או מ-CLI (`python -m tools.run_regression`).
- כך כל בדיקה ידנית שעבדה → מבחן גרסיה קבוע שרץ **לפני** כל מעבר לטלגרם/וואטסאפ.

3. שלב 2 — בוט טלגרם

הוכחה שהמנוע ערוצי-אגנוסטי: adapter דק בלבד.

- **קובץ:** `tools/telegram/telegram_bot.py` (שכבת `tools`, מקביל ל- `tools/whatsapp/`).
- **API:** `backend/app/api/telegram.py` — `POST /api/telegram/webhook`.
זרימה זהה ל-M1000 → `whatsapp: webhook` (אם צריך) / `get_active_session` → `process_message` שליחה חזרה.
- **מיפוי:** כפתורי הבוט (`{id, title}`) → `Telegram inline keyboard`.
לחיצה מחזירה `callback_query` עם `btn.id` → אותן `process_message`.
- **קונפיג:** `TELEGRAM_BOT_TOKEN` ב-env המשותף. `dev: polling (getUpdates)` —
לא דורש כתובת ציבורית; `prod: webhook`. פיצ'ר-פלאג `TELEGRAM_ENABLED`.
- **בדיקה:** אותם סנריואים מסעיף 2.4 — אבל עכשיו מול טלגרם אמיתי (end-to-end ידני).
פרוירטי עדיין `mock (dev)` עד שלב 3.

4. שלב 3 — וואטסאפ (Go-Live)

כבר מחובר ב- `tools/whatsapp/whatsapp_bot.py` + `backend/app/api/whatsapp.py`.
המעבר לפרודקשן הוא בעיקר **הפעלה + הגדרה**, לא קוד חדש:

- [] **הפעלת אינטגרציות אמיתיות** במקום ה-`mock`: כיבוי `BOT_*_MODULE=mock_providers`,
חיבור מודול פרוירטי אמיתי (`equipment reader + service-call writer`).
- [] **ניתוב LLM** = `live` + `OPENAI_API_KEY` תקין.
- [] **Meta creds:** `WHATSAPP_TOKEN`, `PHONE_NUMBER_ID`, `VERIFY_TOKEN`, `webhook URL` (HTTPS).
- [] **אימות webhook** מול `Meta` (handshake — כבר ממומש ב- `whatsapp_verify`).
- [] **מספר בדיקה** אחד לפני שחרור לכולם; מעבר על הסנריואים end-to-end.
- [] **התראות אדמין** (`notify_whatsapp`) / הטלפון 972542777757 בבוט הראשי) — לוודא שעובד.
- [] גיבוי/rollback: אפשר לכבות `WHATSAPP_ENABLED` ולחזור לטלגרם.

5. שדרוגי בדיקה נוספים (מומלצים)

- 🗒 **כיסוי על עורך הזרימה:** אחרי ריצת רגרסיה — לצבוע בעורך הקיים אילו שלבים/ענפים כוסו ומה לא. משתמש ב-`React Flow` שכבר יש. חושף "ענפים מתים".
- 🔍 **בדיקת ניתוב LLM ייעודית:** להריץ את **אותו** צומת הוראות עם 5–10 ניסוחים שונים של ההודעה ולראות אם ה-LLM בוחר את היציאה הנכונה. זה החלק השביר ביותר לפני וואטסאפ. שומר טבלת "ניסוח → יציאה שנבחרה".
- 🧐 **Auto-explore:** מריץ אוטומטית שילוב של כל הכפתורים בכל צומת בחירה ומונה את כל מצבי-הסיום שאפשר להגיע אליהם — מגלה dead-ends ושלבים לא-נגישים.
- 📸 **Snapshot של הודעות:** שמירת הטקסט המדויק שהבוט שולח בכל שלב; רגרסיה מתריעה אם ניסוח השתנה בטעות.

6. סדר עבודה מוצע (אבני דרך)

1. **M1 — תשתית בדיקה (Backend):** `mock_providers.py` + `bot_test.py` + API + refactor ניתוב LLM לפלאגבל. בדיקה עם `curl`.
 2. **M2 — צ'אט אינטראקטיבי (Frontend):** עמוד "בודק שיחות" (3 טורים). בדיקה ידנית מלאה.
 3. **M3 — רגרסיה:** אחסון סנריואים + UI + `save/run/run-all`. בונים 6-8 סנריואים שמכסים את כל הענפים.
 4. **M4 — טלגרם:** `polling` + `webhook` + `adapter` ל-`dev`. בדיקה `end-to-end`.
 5. **M5 — כיסוי + בדיקת LLM** (סעיף 5, אופציונלי לפי צורך).
 6. **M6 — וואטסאפ checklist Go-Live:** סעיף 4.
- מומלץ לעצור לאישור אחרי כל אבן-דרך.

7. שאלות פתוחות / החלטות לפני קוד

1. **בוט ראשי בבדיקה:** האם לבדוק גם את הבוט הראשי (`flow_1772177781916`) עם `switch_script` לבוט התקלה, או להתמקד קודם בבוט התקלה בלבד? (הצ'אט תומך בשניהם; שאלה של סדר עדיפויות).
2. **גיא:** להתעלם ממנו לגמרי כרגע (כפי שביקשת) — לא נכלל בבדיקות. ✓
3. **מספר סנריואים ל-M3:** אצור טיוטה של ~6-8 סנריואים שמכסים את הענפים; תאשר/תוסיף.
4. **טלגרם** `polling dev:` (בלי כתובת ציבורית) מספיק לשלב הבדיקה? (מניח שכן).